Assignment 3: Time-Series Data

BA 64061 — Advanced Machine Learning

Christian Osadebe

1. Purpose

The purpose of this assignment is to apply recurrent neural networks (RNNs) to a time-series problem: the weather time-series forecasting problem. Given a time series of recent weather measurements, the task is to predict the temperature 24 hours in the future. Starting from a stacked recurrent setup, I improve the network using the following methods:

- Adjusting the number of units in each recurrent layer in the stacked setup
- LSTM instead of GRU
- 1D convnet + RNN combined

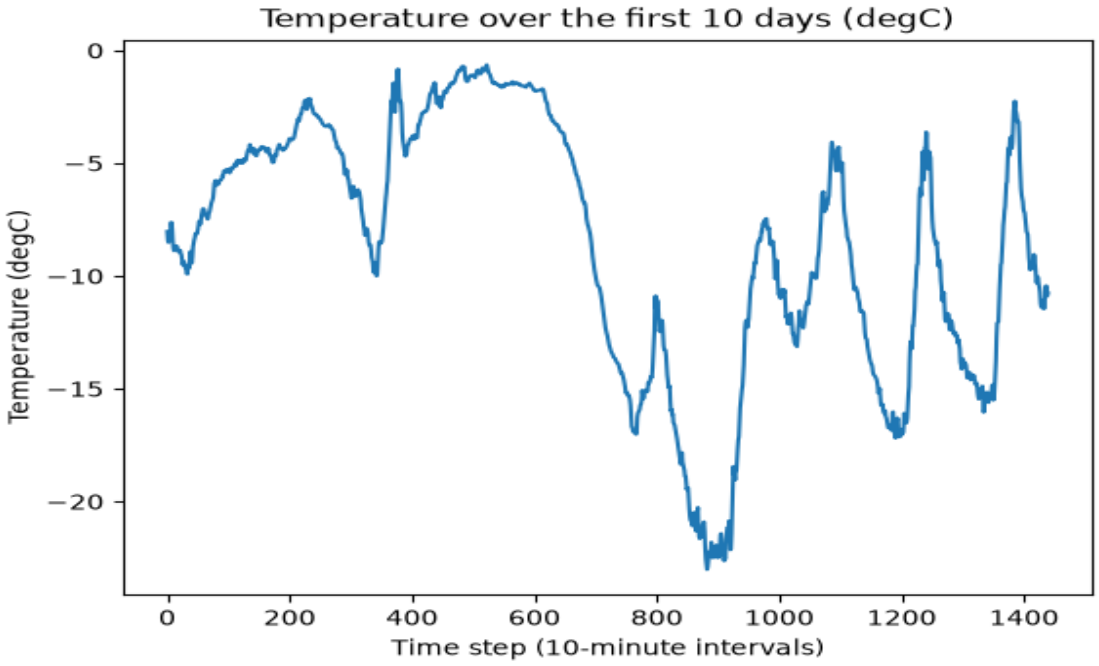
Models are compared by validation MAE, and the best-performing models are then run on the test set. All MAE values below are in degrees Celsius. This report presents the method and the results; the full code is in the accompanying script.

2. The data: the Jena weather dataset

The dataset was recorded at the weather station in Jena, Germany. It contains 14 weather quantities (air temperature, atmospheric pressure, humidity, wind direction, and so on) recorded every 10 minutes. The data used here runs from 2009 to 2016 and has 420,451 rows. The target is the temperature column, T (degC).

Number of training samples: 210,225
Number of validation samples: 105,112
Number of test samples: 105,114

The first 210,225 rows (50%) are used for training, the next 105,112 (25%) for validation, and the remaining 105,114 (25%) for testing.



3. Preparing the data

The data are normalized using statistics computed on the training data only (each column's training mean is subtracted and the result divided by the training standard deviation). Sequences are then built from the data at one point per hour (a sampling rate of 6, since the raw data are recorded every 10 minutes): sequences of 120 points (5 days of hourly data), with a target delay set so the model predicts the temperature 24 hours after the end of the input sequence. The batch size is 256.

samples shape: (256, 120, 14)

targets shape: (256,)

4. A common-sense baseline

The common-sense baseline predicts that the temperature in 24 hours will be the same as the current temperature. Any useful model must beat it.

Validation MAE: 2.44

Test MAE: 2.62

5. Model 1: stacked GRU (baseline recurrent model)

The stacked setup: a first GRU layer with 32 units that returns its full output sequence, a second GRU layer with 32 units, and a dense output layer. It is compiled with the RMSprop optimizer, mean squared error loss, and MAE as the metric, trained for 10 epochs, and the checkpoint from the epoch with the lowest validation MAE is kept for test evaluation.

Best validation MAE: 2.32 (epoch 2)

Test MAE: 2.49

6. Model 2: adjusting the number of units (stacked GRU, 64 → 32)

The same stacked setup, with the first recurrent layer widened from 32 to 64 units. This changed the validation MAE by only 0.01 relative to the stacked GRU baseline (2.31 against 2.32) — the two stacked GRU configurations are effectively tied.

Best validation MAE: 2.31 (epoch 1)

Test MAE: 2.47

7. Model 3: using LSTM instead of GRU (stacked LSTM, 32 → 32)

The same stacked shape with LSTM layers in place of GRU layers. This raised the validation MAE by 0.15 relative to the stacked GRU baseline.

Best validation MAE: 2.47 (epoch 1)

Test MAE: 2.57

8. Model 4: a combination of a 1D convnet and an RNN

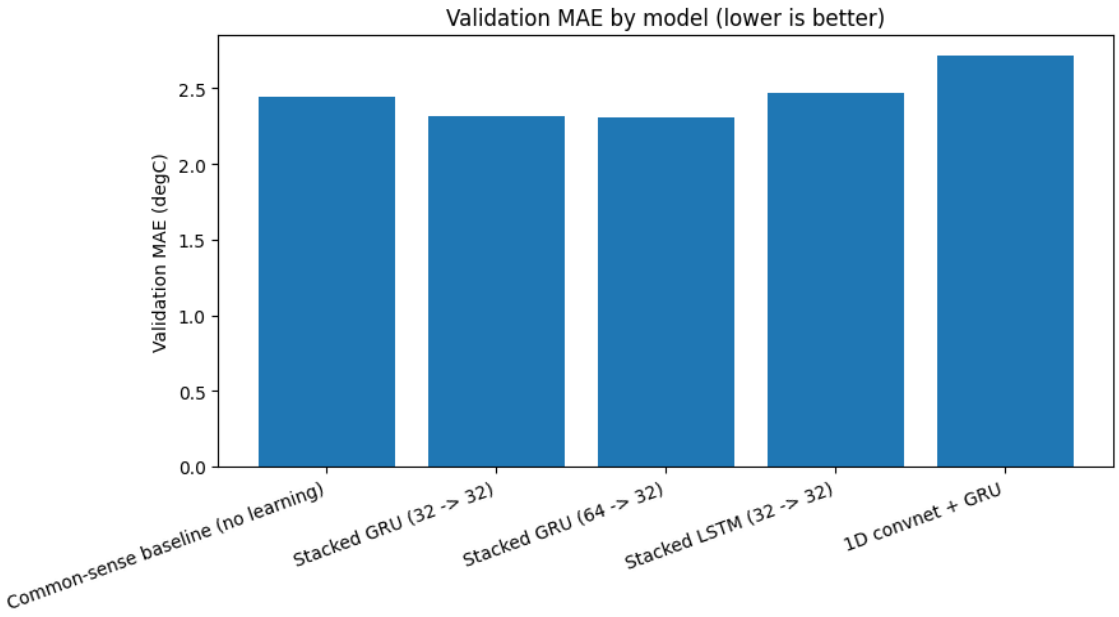
The convolutional front end from the class convnet model (Conv1D layers interleaved with max pooling, which downsamples the 120-step sequence to 13 steps) feeds a GRU layer instead of a global pooling head, so the recurrent layer works on the shorter, convolved sequence. This raised the validation MAE by 0.39 relative to the stacked GRU baseline.

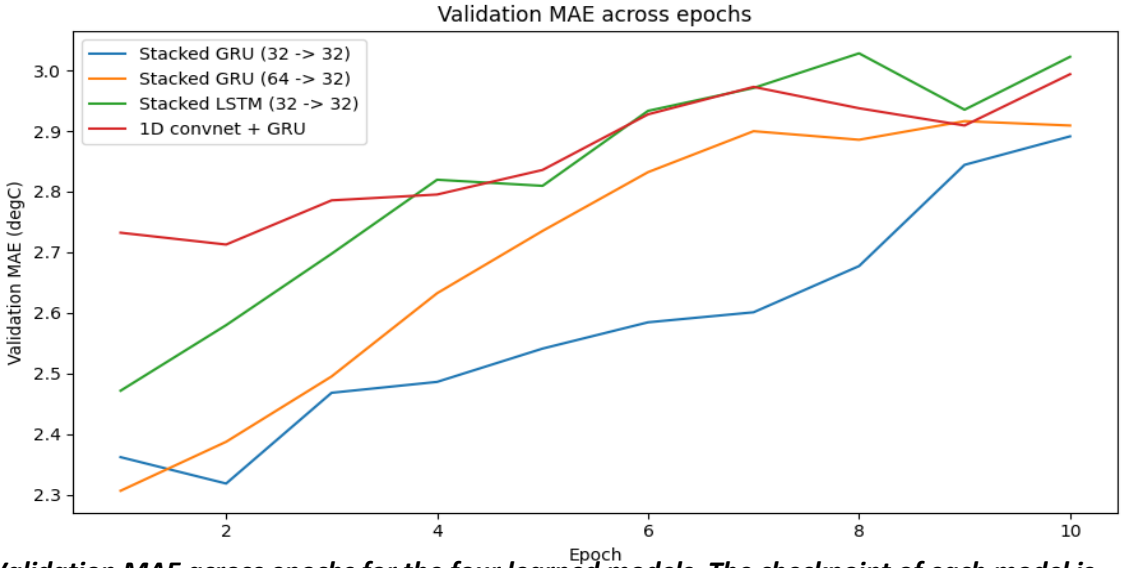
Best validation MAE: 2.71 (epoch 2)

Test MAE: 2.82

9. Summary of results

Model	Validation MAE	Test MAE
Common-sense baseline (no learning)	2.44	2.62
Stacked GRU (32 → 32)	2.32	2.49
Stacked GRU (64 → 32)	2.31	2.47
Stacked LSTM (32 → 32)	2.47	2.57
1D convnet + GRU	2.71	2.82





Validation MAE across epochs for the four learned models. The checkpoint of each model is taken at its best epoch.

10. Conclusions

The common-sense baseline sets the bar at a validation MAE of 2.44 degC, a model is only genuinely useful if it comes in clearly below that. Three of the four learned models do. The best model is Stacked GRU (64 → 32), with a validation MAE of 2.31 degC — a 5.5% improvement over the common-sense baseline — and a test MAE of 2.47 degC, which confirms that its validation performance carries over to unseen data.

Of the three improvement methods tried, none produced a meaningful gain over the plain stacked GRU under this training setup. Widening the first recurrent layer from 32 to 64 units lowered the validation MAE by 0.01 relative to the stacked GRU baseline (2.31 against 2.32) and its test MAE by 0.02 (2.47 against 2.49) — differences within run-to-run noise, so the widened model is best read as tied with the plain stacked GRU rather than better than it; switching from GRU to LSTM in the same stacked shape raised the validation MAE by 0.15 relative to the stacked GRU baseline, and its test MAE (2.57) only just beats the common-sense baseline's (2.62); and putting a 1D convnet in front of the GRU raised the validation MAE by 0.39 relative to the stacked GRU baseline — at this epoch budget it is the one learned model that does not beat the common-sense baseline on either split, although it is the cheapest of the four to train per epoch (about 52 seconds, against about 60 for the stacked GRU), because its GRU only sees the downsampled 13-step sequence.

The training curves tell the underlying story: for the purely recurrent models the best epoch is the first or the second one, and validation MAE rises steadily afterwards while training MAE keeps falling — overfitting. The overall conclusion is that recurrent models do learn real forecasting skill on this problem beyond the naive guess.